LAB 03

Memória no xv6

Tabela de páginas, alocação, falta de página e compartilhamento

Aluno(a)	_____ Data: ____/____/_____

ETAPA 1 /7 Preparar

Abra o ambiente:

```
bash xv6.sh
```

ETAPA 2 /7 Instalar o código (copiar e colar)

Cole os blocos no terminal (xv6), um de cada vez.

1) System calls freemem e pgtprint: números e declarações

```
grep -q SYS_pgtprint kernel/syscall.h || cat >> kernel/syscall.h << 'EOF'
#define SYS_freemem 23
#define SYS_pgtprint 24
EOF
grep -q pgtprint user/user.h || cat >> user/user.h << 'EOF'
int freemem(void);
int pgtprint(void);
EOF
grep -q pgtprint user/usys.pl || cat >> user/usys.pl << 'EOF'
entry("freemem");
entry("pgtprint");
EOF
grep -q vmprint kernel/defs.h || cat >> kernel/defs.h << 'EOF'
uint64 kfreemem(void);
void vmprint(pagetable_t);
EOF
```

2) System calls: tabela e implementação (syscall.c, sysproc.c, kalloc.c)

```
grep -q sys_pgtprint kernel/syscall.c || {
S=kernel/syscall.c
sed -i '/^extern uint64 sys_close/a extern uint64 sys_freemem(void);' $S
sed -i '/^extern uint64 sys_close/a extern uint64 sys_pgtprint(void);' $S
```

```
sed -i '/^\[SYS_close\]/a [SYS_freemem] sys_freemem,' $S
sed -i '/^\[SYS_close\]/a [SYS_pgtprint] sys_pgtprint,' $S
}
grep -q sys_pgtprint kernel/sysproc.c || cat >> kernel/sysproc.c << 'EOF'

uint64
sys_freemem(void)
{
  return kfreemem();
}

uint64
sys_pgtprint(void)
{
  vmprint(myproc()->pagetable); // tabela de quem chamou
  return 0;
}
EOF
grep -q kfreemem kernel/kalloc.c || cat >> kernel/kalloc.c << 'EOF'

// conta as paginas livres percorrendo a lista de paginas livres
uint64
kfreemem(void)
{
  struct run *r;
  uint64 n = 0;
  acquire(&kmem.lock);
  for(r = kmem.freelist; r; r = r->next)
    n++;
  release(&kmem.lock);
  return n * PGSIZE; // paginas -> bytes
}
EOF
```

3) vmprint: imprime a tabela de páginas (vm.c)

```
grep -q vmprint_nivel kernel/vm.c || cat >> kernel/vm.c << 'EOF'

// imprime a tabela de paginas: percorre os 3 niveis (2, 1, 0)
static void
vmprint_nivel(pagetable_t pt, int nivel, uint64 va_base)
{
  for(int i = 0; i < 512; i++){
    pte_t pte = pt[i];
    if((pte & PTE_V) == 0)
      continue; // entrada vazia: pula
    uint64 va = va_base | ((uint64)i << PXSHIFT(nivel));
    uint64 pa = PTE2PA(pte);
    for(int j = nivel; j < 3; j++)
      printf(" ..."); // um .. por nivel
    if(pte & (PTE_R | PTE_W | PTE_X)){ // tem permissao: folha
      char f[5] = "----";
      if(pte & PTE_R) f[0] = 'R';
      if(pte & PTE_W) f[1] = 'W';
      if(pte & PTE_X) f[2] = 'X';
```

```
      if(pte & PTE_U) f[3] = 'U';
      printf("%d: va 0x%lx -> pa 0x%lx [%s]\n", i, va, pa, f);
    } else { // aponta outra tabela
      printf("%d: proxima tabela em pa 0x%lx\n", i, pa);
      vmprint_nivel((pagetable_t)pa, nivel - 1, va);
    }
  }
}

void
vmprint(pagetable_t pt)
{
  printf("tabela de paginas (raiz) em pa 0x%lx\n", (uint64)pt);
  vmprint_nivel(pt, 2, 0);
}
EOF
```

4) user/vertabela.c: mostra a tabela antes e depois de um sbrk

```
cat > user/vertabela.c << 'EOF'
#include "kernel/types.h"
#include "user/user.h"

#define PAG 4096

int
main(int argc, char *argv[])
{
  printf("== ANTES: memoria livre = %d paginas\n", freemem() / PAG);
  pgtprint();

  char *p = sbrk(3 * PAG); // pede 3 paginas novas
  p[0] = 'A'; // usa a memoria nova
  printf("\n== sbrk(3 paginas) devolveu o endereco %p\n", p);

  printf("== DEPOIS: memoria livre = %d paginas\n", freemem() / PAG);
  pgtprint();
  exit(0);
}
EOF
```

5) user/aloca.c: pede N páginas e mede a memória livre

```
cat > user/aloca.c << 'EOF'
#include "kernel/types.h"
#include "user/user.h"

#define PAG 4096

int
main(int argc, char *argv[])
{
  if(argc < 2){
```

```
    printf("uso: aloca <numero de paginas>\n");
    exit(1);
  }
  int n = atoi(argv[1]);
  int antes = freemem() / PAG;
  char *p = sbrk(n * PAG);
  int depois = freemem() / PAG;
  printf("pedi %d paginas a partir de %p\n", n, p);
  printf("livres: %d -> %d (sairam %d)\n", antes, depois, antes-depois);
  exit(0);
}
EOF
```

6) user/falta.c: provoca faltas de página

```
cat > user/falta.c << 'EOF'
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc < 2){
    printf("uso: falta 1 | 2 | 3 | 4 <endereco em decimal>\n");
    exit(1);
  }
  int op = atoi(argv[1]);

  if(op == 1){ // 1) ler um endereco nao mapeado
    int *longe = (int *) 0x40000000; // 1 GB: alem do processo
    printf("lendo 0x40000000...\n");
    printf("valor = %d\n", *longe);
  } else if(op == 2){ // 2) escrever no proprio codigo
    char *codigo = (char *) main; // endereco da funcao main
    printf("escrevendo em main (%p)...\n", codigo);
    codigo[0] = 0;
  } else if(op == 3){ // 3) ler e escrever no heap: ok
    char *p = sbrk(4096);
    p[0] = 'x';
    printf("heap ok: escrevi e li o heap sem problemas\n");
  } else if(op == 4 && argc == 3){ // 4) ler endereco dado
    int *e = (int *) (uint64) atoi(argv[2]);
    printf("lendo %p...\n", e);
    printf("valor = %d\n", *e);
  }
  printf("fim normal do programa\n");
  exit(0);
}
EOF
```

7) user/compartilha.c: compara as tabelas do pai e do filho

```
cat > user/compartilha.c << 'EOF'
#include "kernel/types.h"
```

```
#include "user/user.h"

int
main(int argc, char *argv[])
{
  int pid = fork();
  if(pid == 0){
    printf("\n######## FILHO (pid %d) ########\n", getpid());
    pgtprint();
    exit(0);
  }
  wait(0); // o pai espera o filho imprimir primeiro
  printf("\n######## PAI (pid %d) ########\n", getpid());
  pgtprint();
  exit(0);
}
EOF
```

8) user/memlivre.c: mede o gasto do sbrk e do fork

```
cat > user/memlivre.c << 'EOF'
#include "kernel/types.h"
#include "user/user.h"

#define PAG 4096

int
main(int argc, char *argv[])
{
  int m0 = freemem();
  printf("[1] inicio: %d pags livres\n", m0/PAG);

  sbrk(10 * PAG); // pede 10 paginas a mais
  int m1 = freemem();
  printf("[2] sbrk: %d livres (-%d)\n", m1/PAG, (m0-m1)/PAG);

  int pid = fork();
  if(pid == 0){
    int m2 = freemem();
    printf("[3] filho: %d livres (-%d)\n", m2/PAG, (m1-m2)/PAG);
    exit(0);
  }
  wait(0);
  int m3 = freemem();
  printf("[4] fim: %d livres (-%d desde [2])\n", m3/PAG, (m1-m3)/PAG);
  exit(0);
}
EOF
```

9) Registrar os 5 programas no Makefile

```
grep -q _vertabela Makefile ||
  sed -i '/_zombie/a\\t$U/_vertabela\\' Makefile
grep -q _aloca Makefile || sed -i '/_zombie/a\\t$U/_aloca\\' Makefile
```

```
grep -q _falta Makefile || sed -i '/_zombie/a\\t$U/_falta\\' Makefile
grep -q _compartilha Makefile ||
  sed -i '/_zombie/a\\t$U/_compartilha\\' Makefile
grep -q _memlivre Makefile || sed -i '/_zombie/a\\t$U/_memlivre\\' Makefile
```

```
make qemu
```

✓*Pronto quando: aparece o prompt $.*

ETAPA 3 /7 Tabela de páginas

```
$ vertabela
```

Resultado esperado

```
tabela de paginas (raiz) em pa 0x87f25000
 ..0: proxima tabela em pa 0x87f21000
 .. ..0: proxima tabela em pa 0x87f20000
 .. .. ..0: va 0x0 -> pa 0x87f22000 [R-XU]
 ...
```

Flag	Significado
R / W / X	pode ler / escrever / executar
U	o programa do usuário pode acessar

Use a 1ª tabela (ANTES). Região: código, dados, guarda, pilha, trapframe ou trampoline.

va	Flags	Região
0x0	*R-XU*	*Código*
0x1000	*RW-U*	*Dados*
0x2000	*RW--*	*Página de guarda*
0x3000	*RW-U*	*Pilha*
0x3fffffe000	*RW--*	*Trapframe*
0x3ffffff000	*R-X-*	*Trampoline*

✍ **SUAS OBSERVAÇÕES**

a) Quantas páginas de tabela (linhas “proxima tabela”) o processo usa?
b) Por que a página 0x2000 não tem a flag U?
c) No bloco 3 (vmprint), qual linha faz a função descer um nível?

a) Na saída aparecem quatro linhas *proxima tabela*: duas para endereços virtuais baixos e duas para os endereços virtuais altos.

b) Porque é uma página de guarda, utilizada para proteger a pilha. Sem a flag *U*, o processo em modo usuário não pode acessar essa página, permitindo detectar acessos inválidos, como o crescimento da pilha para além do espaço permitido.

c) É a chamada recursiva de *vmprint* (ou da função auxiliar responsável pela impressão), passando a tabela de páginas do próximo nível. (*vmprint_nivel((pagetable_t)pa, nivel - 1, va);).*

ETAPA 4 /7 Alocação

Compare a 2ª tabela do vertabela (DEPOIS do sbrk) com a 1ª. Depois preveja e rode:

Comando	Previsão (páginas)	Saíram (observado)
`aloca 1`	*1*	*1*
`aloca 3`	*3*	*3*
`aloca 600`	*601*	*601*

✍ **SUAS OBSERVAÇÕES**

Por que o aloca 600 gastou uma página a mais? (Uma tabela tem 512 entradas.)

Porque cada tabela de último nível possui 512 entradas. Ao solicitar 600 páginas, o processo ultrapassou a capacidade da tabela existente, sendo necessário criar outra página de tabela para mapear novos endereços.

ETAPA 5 /7 Falta de página

A) Mensagem original do kernel

```
$ falta 1
```

	Valor
scause	0xd pid=8
stval	stval=0x40000000

B) Mensagem nova (copiar e colar)

Desligue o xv6 (Ctrl-a, x) e cole o bloco. Ele cria a função faltapagina() em kernel/trap.c e a chama em usertrap():

```
grep -q faltapagina kernel/trap.c || {
echo 'int faltapagina(struct proc*);' >> kernel/defs.h
T=kernel/trap.c
sed -i '/else if((which_dev = devintr/i\ } else if(faltapagina(p)){' $T
cat >> $T << 'EOF'

// falta de pagina? (scause 13 = leitura, 15 = escrita)
int
faltapagina(struct proc *p)
{
  uint64 c = r_scause();
  if(c != 13 && c != 15)
    return 0; // nao e falta de pagina
  printf("FALTA DE PAGINA: processo %d (%s) tentou %s",
         p->pid, p->name, c == 13 ? "LER" : "ESCREVER");
  printf(" o endereco 0x%lx\n", r_stval());
  setkilled(p); // mata o processo
  return 1;
}
EOF
}
```

```
make qemu
```

Preveja e depois rode no xv6:

Comando	O que faz	Previsão	Observado
`falta 1`	lê 0x40000000	*Falta de leitura*	*FALTA DE PAGINA: LER*
`falta 2`	escreve em main (0x0)	*Falta de escrita*	*FALTA DE PAGINA: ESCREVER*
`falta 3`	usa o heap	*Execução normal*	*Executou normalmente*
`falta 4 4096`	lê 0x1000	*Leitura permitida*	*Valor = 0*
`falta 4 8192`	lê 0x2000	*Falta de leitura*	*FALTA DE PAGINA: LER*

✍ **SUAS OBSERVAÇÕES**

A página 0x0 existe. Por que o falta 2 deu falta de página?

Porque a página *0x0* pertence à região de código e possui as permissões *R-XU* (leitura, execução e acesso pelo usuário), mas não possui permissão de escrita (*W*).

ETAPA 6 /7 Compartilhamento

```
$ compartilha
```

va	pa FILHO	pa PAI	Igual?
`0x0`	*0x87f40000*	*0x87f54000*	*Não*
`0x1000`	*0x87f2c000*	*0x87f1d000*	*Não*
`0x3fffffe000`	*0x87f1c000*	*0x87f34000*	*Não*
`0x3ffffff000`	*0x80006000*	*0x80006000*	*Sim*

```
$ memlivre
```

✍ **SUAS OBSERVAÇÕES**

Qual página é compartilhada? Como o copy-on-write reduziria o gasto do fork?

A página *tampoline*, no endereço virtual *0x3ffffff000*, é compartilhada entre pai e filho. Ela contém código utilizado nas transições entre modo usuário e o modo kenel. O *copy-on-write* (COW) permitiria que pai e filho dividissem inicialmente as mesmas páginas físicas de memória, em vez de copiar todas imediatamente durante o *fork()*.

ETAPA 7 /7 Síntese

Conceito	Onde você viu (programa + observação)
Tabela de páginas	*vertabela* - mostrou os endereços virtuais, físicos e as permissões de cada página.
Alocação	*aloca* - ao pedir 600 páginas, foram utilizadas 601, pois foi necessária uma página adicional para a tabela.

Conceito	Onde você viu (programa + observação)
Falta de página	*falta* – mostrou erros ao acessar endereços não mapeados ou sem permissão.
Compartilhamento	*compartilha* - mostrou que pai e filho compartilham a página *trampoline*, enquanto outras páginas são copiadas.
Proteção (flags)	*vertabela* e *falta* - as flags *R*, *W*, *X* e *U* controlam os acessos. A tentativa de escrever no código gerou uma falta de página.